

SBSE no contexto de microsserviços

Hélio Toshio Kamakawa Kananda Caroline

Programa de Pós-Graduação em Ciência da Computação – Universidade Estadual de Maringá

Tópicos Avançados em Engenharia de Software II

Professores: Thelma Elita Colanzi e João Choma Neto

Resumo—A identificação de fronteiras de microsserviços é uma decisão arquitetural complexa, pois exige particionar elementos de um sistema segundo objetivos potencialmente conflitantes em um espaço combinatório de soluções. Este relatório analisa a aplicação de *Search-Based Software Engineering* (SBSE) a esse problema por meio de dois estudos explicitamente baseados em busca: o **MSExtractor**, que emprega IBEA para decompor classes de sistemas legados, e o **SAMIM**, que utiliza MOEA/D para identificar microsserviços variáveis a partir de processos de negócio SaaS. Como estudo complementar, examina-se o método de Kinoshita e Kanuka, apresentado pelos autores como otimização multiobjetivo para refinar uma decomposição inicial mediante políticas, detecção de violações e operações de correção. A comparação considera as entradas, a representação das soluções, as funções objetivo, a geração e a avaliação de candidatos e os mecanismos de seleção. Apesar dos diferentes contextos e métodos, a análise permite retomar uma estrutura fundamental: formular uma decisão de Engenharia de Software como um problema de otimização, definir seu espaço de busca e a representação das soluções, traduzir propriedades desejáveis em funções objetivo e explorar sistematicamente as alternativas. Nas formulações multiobjetivo, objetivos conflitantes tornam explícitos os *trade-offs* e conduzem à identificação de soluções não dominadas que formam ou aproximam uma fronteira de Pareto. **MSExtractor** e **SAMIM** constituem formulações evolutivas distintas de SBSE; o terceiro método compartilha elementos dessa estrutura de modelagem e otimização, mas não é denominado explicitamente SBSE no artigo. Em todos os casos, a otimização organiza a exploração das alternativas e apoia a decisão arquitetural, sem substituir a escolha contextual e o julgamento do arquiteto.

I. CONTEXTO E MOTIVAÇÃO

Sistemas legados podem concentrar lógica, dados e funções em uma aplicação monolítica, o que torna mais difícil alterar uma funcionalidade isoladamente à medida que o sistema cresce [3]. A arquitetura de microsserviços procura distribuir essas responsabilidades em serviços pequenos, coesos e fracamente acoplados, capazes de ser desenvolvidos, implantados e escalados de modo independente [1]. A migração, entretanto, não consiste apenas em reduzir o tamanho da aplicação: uma decomposição inadequada pode preservar dependências indesejadas e até degradar a qualidade arquitetural do sistema original [1], [3].

Nesse contexto, a migração de sistemas monolíticos para arquiteturas de microsserviços apresenta um problema fundamental: determinar *onde estabelecer as fronteiras dos serviços*. Sellami et al. [1] caracterizam a identificação dessas fronteiras como a primeira e uma das mais difíceis etapas da migração. A decisão envolve definir quais classes, módulos e, quando considerados, elementos de banco de dados devem permanecer

juntos e quais devem ser separados. Portanto, ela influencia diretamente a estrutura da arquitetura resultante.

A dificuldade dessa decisão decorre, entre outros fatores, da existência de critérios potencialmente conflitantes. Neste relatório, **coesão** designa o grau de relação entre elementos agrupados no mesmo serviço; **acoplamento**, as dependências que atravessam fronteiras entre serviços; e **granularidade**, a organização dos elementos quanto ao tamanho e à quantidade dos serviços resultantes. No **MSExtractor**, esses conceitos são operacionalizados, respectivamente, pelas dependências internas aos serviços, pelas dependências entre serviços e por uma função destinada a controlar a granularidade da decomposição [1]. Buscar simultaneamente alta coesão, baixo acoplamento e granularidade adequada exige analisar compromissos, e não apenas maximizar isoladamente uma única propriedade.

Outro fator relevante é o crescimento combinatorial do número de decomposições possíveis. Cada forma de particionar os elementos de um sistema representa uma solução candidata e, mesmo para sistemas relativamente pequenos, o número dessas partições pode tornar inviável uma avaliação exaustiva. Sellami et al. [1], por exemplo, tratam a identificação de microsserviços em sistemas legados orientados a objetos como um problema de otimização combinatorial multiobjetivo, no qual diferentes decomposições precisam ser exploradas e avaliadas.

Diante desse cenário, a identificação de fronteiras de microsserviços pode ser formulada como um problema de otimização. Essa perspectiva permite empregar técnicas de *Search-Based Software Engineering* (SBSE), nas quais algoritmos de busca exploram grandes espaços de soluções e procuram compromissos adequados entre objetivos. Neste relatório, esse enquadramento é examinado por meio de duas propostas automatizadas. A primeira é o **MSExtractor**, abordagem de Sellami et al. [1] para identificar candidatos a microsserviços em sistemas legados orientados a objetos mediante busca evolutiva. A segunda é o **SAMIM**, método de Khoshnevis [2] para identificar microsserviços variáveis em aplicações SaaS a partir de modelos de processos de negócio. O trabalho de Kinoshita e Kanuka [3] é incluído posteriormente como abordagem complementar de otimização multiobjetivo: ele compartilha parte da estrutura conceitual, mas não identifica diretamente o método proposto como SBSE.

II. O QUE É SBSE?

Search-Based Software Engineering (SBSE) consiste em formular um problema de Engenharia de Software como um problema de busca e otimização. Essa formulação requer: uma representação das soluções candidatas; uma ou mais funções objetivo, ou *fitness*, que expressem as propriedades desejadas; e um mecanismo de busca que produza e compare alternativas. Em vez de avaliar exaustivamente todas as possibilidades, o algoritmo utiliza as funções objetivo para orientar a exploração do espaço de soluções. Essa estrutura é observada explicitamente na decomposição de classes com IBEA [1] e na decomposição de atividades com MOEA/D [2].

No contexto da identificação de fronteiras de microsserviços, uma solução candidata representa uma possível decomposição do sistema. Sellami et al. [1] representam decomposições de sistemas orientados a objetos e avaliam coesão, acoplamento e granularidade. Khoshnevis [2] utiliza otimização multiobjetivo para identificar microsserviços considerando a variabilidade de aplicações SaaS. Esses dois estudos constituem os exemplos centrais de SBSE do relatório. Posteriormente, o método de Kinoshita e Kanuka [3] será contrastado com eles por também representar, gerar e avaliar alternativas, embora sua proposta seja apresentada pelos autores como otimização multiobjetivo e não explicitamente como SBSE.

A. Espaço de busca

O **espaço de busca** corresponde ao conjunto das soluções candidatas admitidas pela representação do problema. Na identificação de fronteiras de microsserviços, cada solução corresponde a uma decomposição completa do sistema, isto é, uma determinada forma de distribuir seus elementos entre diferentes microsserviços. O tamanho desse espaço depende tanto da quantidade de elementos quanto das restrições adotadas. Khoshnevis [2], por exemplo, ilustra que mesmo sete atividades, quando cada uma pode ser atribuída a até sete serviços, já originam centenas de milhares de codificações possíveis.

Uma maneira de representar uma decomposição é utilizar um vetor de inteiros. Cada posição do vetor corresponde a um elemento do sistema e o valor armazenado nessa posição identifica o microsserviço ao qual esse elemento pertence. Sellami et al. [1] empregam esse princípio para representar a atribuição das classes de uma aplicação orientada a objetos aos microsserviços candidatos. Assim, diferentes vetores representam diferentes possibilidades de decomposição e, consequentemente, diferentes pontos no espaço de busca.

Além da representação das soluções, um problema de SBSE requer **funções objetivo** (*fitness*) que permitam avaliar as alternativas encontradas. Sellami et al. [1] utilizam coesão, acoplamento e granularidade; Khoshnevis [2] considera convergência, granularidade e comunalidade. Na abordagem complementar, Kinoshita e Kanuka [3] também adotam múltiplas funções, mas elas medem violações de políticas arquiteturais.

O terceiro componente é o **algoritmo de busca**, responsável por explorar o espaço de soluções utilizando as funções objetivo como orientação. Essa exploração é necessária porque

o número de decomposições possíveis cresce rapidamente, tornando impraticável testar todas as alternativas. Em vez de realizar uma busca exaustiva, os algoritmos procuram regiões promissoras do espaço e produzem soluções candidatas com diferentes características.

A Figura 1 apresenta um exemplo simplificado dessa representação. Cada posição do vetor corresponde a uma classe, enquanto o valor associado indica o microsserviço ao qual ela foi atribuída.

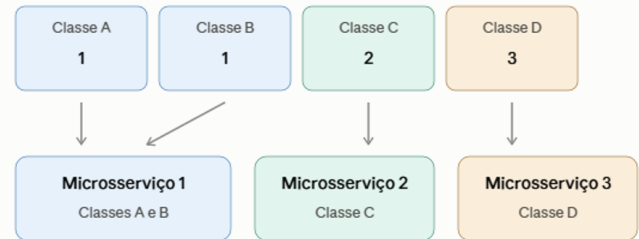


Figura 1. Exemplo de representação de uma solução no espaço de busca. Fonte: elaboração própria.

No exemplo da Figura 1, o vetor $[1, 1, 2, 3]$ representa uma decomposição completa: as classes A e B pertencem ao microsserviço 1, a classe C ao microsserviço 2 e a classe D ao microsserviço 3. Portanto, uma alteração nos valores do vetor produz outra forma de dividir o mesmo sistema e, consequentemente, outra solução dentro do espaço de busca. Essa representação torna possível aplicar algoritmos de otimização para explorar sistematicamente diferentes decomposições.

B. Objetivos conflitantes e Fronteira de Pareto

A identificação de fronteiras de microsserviços não envolve necessariamente um único objetivo. Diferentes propriedades arquiteturais podem ser desejáveis simultaneamente e, em determinadas situações, a melhoria de uma delas pode ocorrer às custas de outra. Um exemplo é a relação entre **coesão** e **acoplamento**: busca-se manter fortemente relacionados os elementos que pertencem ao mesmo microsserviço e, ao mesmo tempo, reduzir dependências entre microsserviços distintos.

Sellami et al. [1] formulam explicitamente a extração de microsserviços como um problema de otimização multiobjetivo, considerando simultaneamente características relacionadas à coesão, ao acoplamento e à granularidade das soluções. De forma semelhante, Khoshnevis [2] utiliza múltiplos objetivos na identificação de microsserviços para aplicações SaaS. Dessa forma, não se procura necessariamente uma única solução que seja superior em todos os critérios, mas diferentes alternativas que representem compromissos entre os objetivos.

Esse cenário conduz ao conceito de **dominância de Pareto**. Considerando objetivos expressos na mesma direção de preferência, uma solução domina outra quando não é pior em nenhum objetivo e é estritamente melhor em pelo menos um. As soluções que não são dominadas formam a denominada **Fronteira de Pareto**. Cada solução nessa fronteira representa, portanto, um compromisso diferente entre os objetivos avaliados. Kinoshita e Kanuka [3], por exemplo, utilizam ordenação

não dominada para preservar decomposições avaliadas simultaneamente por quatro políticas arquiteturais.

Algoritmos evolutivos multiobjetivo (*Multi-Objective Evolutionary Algorithms* – MOEAs) são empregados para explorar esse tipo de problema. Os trabalhos analisados utilizam diferentes estratégias: Sellami et al. [1] empregam o IBEA e também o comparam experimentalmente com NSGA-II e SPEA2; Khoshnevis [2] utiliza o MOEA/D. Em ambos os casos, a otimização permite explorar múltiplas alternativas em vez de pressupor uma única decomposição ideal.

Assim, o resultado da busca pode oferecer ao arquiteto um conjunto de alternativas com diferentes compromissos entre os objetivos. A otimização funciona, nesse sentido, como mecanismo de apoio à decisão arquitetural, não eliminando a necessidade de avaliar qual alternativa é mais adequada ao contexto do sistema.

C. Fronteira de Pareto: coesão $\times$ acoplamento

A relação entre coesão e acoplamento permite visualizar de maneira simplificada o funcionamento da Fronteira de Pareto. Considerando que se deseja **aumentar a coesão** dentro dos microserviços e **reduzir o acoplamento** entre microserviços distintos, diferentes decomposições podem apresentar vantagens em um objetivo e desvantagens no outro.

A Figura 2 ilustra esse cenário. Cada ponto representa uma solução candidata avaliada segundo os dois objetivos. Os pontos destacados na fronteira correspondem às soluções não dominadas, enquanto os demais representam soluções para as quais existe pelo menos uma alternativa superior segundo os critérios considerados.

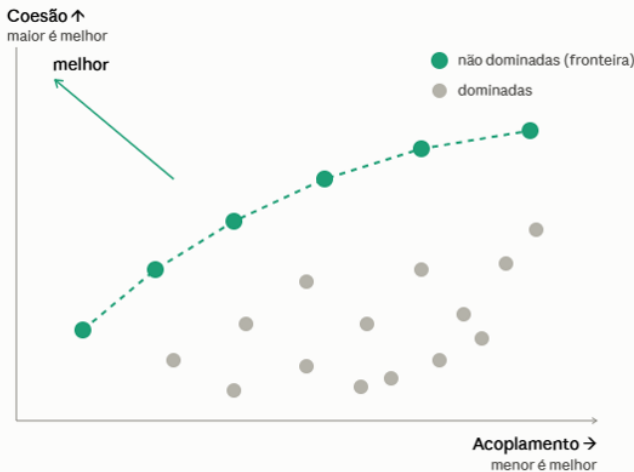


Figura 2. Exemplo de Fronteira de Pareto considerando coesão e acoplamento. Fonte: elaboração própria.

Como sintetizado na Figura 2, pertencer à Fronteira de Pareto não significa que uma solução seja, isoladamente, “a melhor”. Significa que ela representa um compromisso para o qual não existe outra solução que a supere simultaneamente nos objetivos considerados. Essa característica é relevante no MSExtractor e no SAMIM, que aplicam SBSE

multiobjetivo [1], [2]. O método complementar também utiliza dominância de Pareto para comparar alternativas geradas por correções arquiteturais [3].

III. MONÓLITO $\times$ MICROSERVIÇOS

Uma aplicação monolítica concentra diferentes lógicas, dados e funções em uma mesma aplicação, de modo que a alteração de uma funcionalidade pode exigir compreender e modificar partes relacionadas do conjunto [3]. Essa organização contrasta com a arquitetura de microserviços, mas não implica, por si só, baixa qualidade: o problema relevante para a migração é identificar quais dependências precisam ser reorganizadas e quais limites arquiteturais são adequados ao contexto.

A arquitetura de microserviços adota uma organização baseada em serviços menores, com interfaces bem definidas e comunicação por mecanismos leves. Esses serviços procuram manter alta coesão e baixo acoplamento e podem ser implantados independentemente [1], [2]. Em algumas formulações, cada serviço também executa uma função ou capacidade específica de negócio e mantém sua própria lógica e seus dados [3]. Consequentemente, a definição das fronteiras entre esses serviços torna-se uma decisão arquitetural central.

Essa mudança, entretanto, não consiste apenas em separar o código de uma aplicação. Os dados também fazem parte das fronteiras arquiteturais. A autonomia de um serviço envolve os componentes de software e os dados pelos quais ele é responsável. No modelo de Kinoshita e Kanuka [3], essa responsabilidade é operacionalizada pela propriedade da tabela: acessos de escrita — criação, atualização e exclusão — indicam qual microserviço é responsável pelo respectivo modelo de domínio.

A independência proporcionada pela decomposição também introduz novos desafios. Operações que em um monólito ocorriam dentro de um único processo e sobre um banco de dados compartilhado podem, após a decomposição, exigir comunicação pela rede e coordenação entre diferentes serviços e fontes de dados. Assim, benefícios como escalabilidade granular, autonomia das equipes e isolamento de falhas são acompanhados por problemas característicos de sistemas distribuídos, como latência, comunicação remota e manutenção da consistência dos dados.

Essa relação entre decomposição e dados é particularmente importante para os trabalhos analisados. Sellami et al. [1] realizam a decomposição a partir de informações estruturais do código, sem incorporar explicitamente o banco de dados aos objetivos de otimização. Khoshnevis [2] considera entidades de negócio e os acessos realizados pelas atividades do processo. Kinoshita e Kanuka [3], por sua vez, incorporam explicitamente elementos de persistência e operações de acesso aos dados em seu modelo de decomposição. Essas diferenças permitem observar diferentes níveis de consideração dos dados durante a identificação das fronteiras de microserviços.

IV. BANCO DE DADOS COMO DIMENSÃO DE ACOPLAMENTO

A separação adequada do código não garante, isoladamente, a independência entre microserviços. Se serviços distintos permanecerem fortemente dependentes das mesmas estruturas de persistência, parte do acoplamento existente no monólito é preservada na camada de dados. Por esse motivo, o banco de dados constitui uma dimensão relevante na definição das fronteiras dos serviços.

Um primeiro aspecto é a **propriedade dos dados**. Em uma decomposição orientada à autonomia, cada microserviço deve ser responsável pelo modelo de dados associado à sua função. Kinoshita e Kanuka [3] traduzem esse princípio em uma política verificável: cada tabela deve receber acessos de escrita de apenas um microserviço. Quando serviços diferentes escrevem diretamente nas mesmas tabelas, a responsabilidade sobre os dados torna-se compartilhada e alterações na persistência podem afetar mais de um serviço.

Um segundo aspecto envolve as **transações distribuídas**. Uma transação originalmente contida no monólito pode ser dividida entre serviços após a decomposição. Nesse caso, mecanismos como Saga ou *Try-Commit-Cancel* podem ser necessários, acrescentando código de coordenação que não pertence diretamente à lógica de negócio [3]. Portanto, uma decomposição realizada sem considerar as dependências de dados pode introduzir coordenação transacional entre serviços que deveriam ser independentes.

Essa preocupação aparece explicitamente no trabalho de Kinoshita e Kanuka [3]. Entre as políticas arquiteturais consideradas pelos autores estão a restrição de acesso de escrita a tabelas por diferentes microserviços e a prevenção de transações distribuídas. Dessa maneira, características relacionadas ao banco de dados deixam de ser apenas consequências da decomposição e passam a participar da avaliação das soluções candidatas.

Um terceiro aspecto são os **relacionamentos do esquema que atravessam potenciais fronteiras de serviços**. Uma chave estrangeira expressa uma relação de integridade entre tabelas; se essas tabelas forem atribuídas a serviços distintos, a relação passa a exigir uma decisão explícita de projeto, como comunicação entre serviços ou replicação controlada de informações. Os três métodos analisados não utilizam chaves estrangeiras como função objetivo. Assim, neste relatório, elas são tratadas apenas como uma possível evidência estática de dependência, e não como critério validado pelos experimentos discutidos.

Uma característica particularmente interessante dessas informações é que parte delas pode ser obtida diretamente da estrutura do sistema. Tabelas, colunas, relações e operações de acesso aos dados podem constituir evidências objetivas para avaliar uma decomposição. Kinoshita e Kanuka [3], por exemplo, representam explicitamente banco de dados, tabelas, colunas e operações de acesso CRUD em seu metamodelo, permitindo que violações relacionadas aos dados sejam consideradas durante a otimização.

Os três trabalhos analisados evidenciam abordagens distintas para essa dimensão. No MSExtractor, de Sellami et al. [1], os objetivos utilizados na busca são derivados das dependências do código e o banco de dados não participa diretamente da formulação da otimização. No SAMIM, Khoshnevis [2] relaciona atividades às entidades de negócio acessadas e utiliza essa informação no cálculo da convergência. Já Kinoshita e Kanuka [3] incorporam elementos concretos de persistência e acessos CRUD à representação da solução. Essa evolução será retomada na análise individual dos trabalhos.

V. O PROBLEMA DE IDENTIFICAÇÃO DE FRONTEIRAS

A identificação de fronteiras de microserviços pode ser entendida como um problema de particionamento: os elementos existentes em um sistema legado — como classes, módulos e, dependendo da abordagem, elementos relacionados aos dados — precisam ser distribuídos entre serviços candidatos. Cada distribuição possível representa uma alternativa arquitetural diferente.

É importante distinguir esse processo de uma tarefa de descoberta ou seleção de serviços já disponíveis. Khoshnevis [2] diferencia explicitamente a descoberta de serviços existentes da identificação, entendida como o projeto de novos serviços a partir de requisitos ou artefatos do sistema. Em uma aplicação monolítica, portanto, os microserviços não estão necessariamente prontos para serem apenas encontrados. A decomposição envolve **construir uma organização arquitetural**: decidir quais elementos permanecerão juntos e quais serão separados. Trata-se de um problema de projeto no qual diferentes alternativas podem ser válidas segundo diferentes critérios.

Uma das principais dificuldades é o tamanho do espaço de busca. O número de maneiras de particionar um conjunto de n elementos é dado pelo **número de Bell** B_n . Os primeiros valores da sequência são

1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, ...

Isso significa que apenas dez elementos já admitem 115975 partições possíveis. Para vinte elementos, o número ultrapassa 51 trilhões de possibilidades. O crescimento é, portanto, combinatorial e torna rapidamente inviável avaliar todas as decomposições por força bruta.

Essa dimensão torna-se ainda mais evidente quando comparada aos sistemas utilizados nos estudos. Na avaliação de Sellami et al. [1], por exemplo, os sistemas analisados possuem de dezenas a centenas de classes, chegando a *Roller*, com 534 classes. Considerar exaustivamente todas as formas possíveis de agrupar centenas de elementos seria impraticável.

É justamente nesse ponto que SBSE se torna relevante. Em vez de enumerar todas as partições, define-se uma representação, estabelecem-se objetivos e utiliza-se um algoritmo de busca para explorar seletivamente as alternativas. Sellami et al. [1] e Khoshnevis [2] realizam essa exploração por algoritmos evolutivos. Kinoshita e Kanuka [3], por sua vez, geram alternativas por correções predefinidas e as filtram

por dominância, oferecendo um contraste útil com as duas formulações explícitas de SBSE.

Assim, o problema central não é simplesmente produzir uma divisão do sistema, mas encontrar, dentro de um espaço combinatorialmente grande, decomposições com propriedades arquiteturais desejáveis. Os dois estudos seguintes mostram formas distintas de aplicar SBSE a esse problema; o terceiro amplia a comparação com outra formulação de otimização multiobjetivo.

VI. MSEXTRACTOR – SELLAMI ET AL. (2022)

Sellami et al. [1] propõem o MSExtractor, uma abordagem destinada a identificar automaticamente possíveis fronteiras de microserviços em aplicações legadas orientadas a objetos. A expressão *possíveis fronteiras* é importante: a abordagem não pressupõe a existência de uma única decomposição correta, mas formula o problema como uma otimização combinatoria multiobjetivo, na qual diferentes decomposições podem representar compromissos distintos entre os objetivos considerados.

O trabalho foi publicado em 2022 no periódico *Information and Software Technology*. Para o recorte deste relatório, uma característica particularmente relevante é que a formulação do MSExtractor é baseada em informações estruturais do código. O banco de dados não é utilizado como entrada nem participa diretamente das funções objetivo empregadas durante a busca [1].

A. Formulação SBSE

No MSExtractor, cada solução candidata representa uma decomposição completa das classes da aplicação em microserviços. Essa solução é codificada por um vetor de inteiros: cada posição corresponde a uma classe e o valor armazenado indica o microserviço ao qual a classe foi atribuída [1]. Dessa forma, diferentes atribuições das classes produzem diferentes soluções no espaço de busca.

O modelo distingue ainda classes *inner*, utilizadas internamente por determinado microserviço, e classes *interface*, que estabelecem comunicação com outros microserviços. Essa distinção é relevante para o cálculo do acoplamento, pois as classes de interface representam pontos de dependência entre os serviços candidatos [1].

A qualidade das soluções é avaliada por três funções objetivo, todas calculadas a partir de uma medida de **similaridade entre classes** $CS(c_i, c_j)$, definida como combinação ponderada de similaridade estrutural (Sim^{STR} , baseada em chamadas de métodos compartilhados) e similaridade semântica (Sim^{SEM} , baseada em vetores tf-idf do vocabulário do código-fonte):

$$CS(c_i, c_j) = \alpha Sim^{STR}(c_i, c_j) + \beta Sim^{SEM}(c_i, c_j), \\ \alpha + \beta = 1.$$

Na configuração experimental, os autores adotam $\alpha = \beta = 0,5$ [1]. Os três objetivos são definidos sobre essa medida:

Coesão. A coesão de um microserviço m é calculada como o complemento da média de CS entre todos os pares de classes

pertencentes a m . A coesão da decomposição $\mathcal{M}$ agrega esse valor para todos os microserviços:

$$Cohesion(\mathcal{M}) = 1 - \frac{\sum_{m \in \mathcal{M}} Coh(m)}{|\mathcal{M}|}, \quad \text{a ser maximizada}$$

Acoplamento. O acoplamento entre dois microserviços m_1 e m_2 é calculado exclusivamente sobre suas classes de *interface* (I_1 e I_2), pois são elas que expõem funcionalidades para outros serviços. As classes *inner* são excluídas por não participarem da comunicação entre microserviços:

$$C(m_1, m_2) = \frac{\sum_{\forall c_i \in I_1, \forall c_j \in I_2} CS(c_i, c_j)}{|I_1| \times |I_2|}$$

O acoplamento da decomposição $Coupling(\mathcal{M})$ é a média de $C(m_i, m_j)$ sobre todos os pares de microserviços distintos, a ser **minimizado** [1].

Granularidade. A granularidade é a razão entre o número total de classes $|C|$ e o número de microserviços $|\mathcal{M}|$:

$$Granularity(\mathcal{M}) = \frac{|C|}{|\mathcal{M}|}.$$

Essa função é minimizada dentro da faixa configurável definida para o tamanho dos microserviços.

Essas três propriedades são calculadas a partir das dependências existentes no código-fonte, sem incorporar o esquema ou os acessos ao banco de dados [1].

A busca é realizada principalmente pelo IBEA (*Indicator-Based Evolutionary Algorithm*), algoritmo evolutivo multiobjetivo que utiliza indicadores de qualidade para orientar a seleção das soluções. Na configuração experimental apresentada pelos autores, foram utilizados uma população de 100 indivíduos, até 2.000 gerações, probabilidade de cruzamento de 0,8 e probabilidade de mutação de 0,1. Para avaliar a escolha do algoritmo, o IBEA também foi comparado com NSGA-II e SPEA2 sob condições equivalentes [1].

B. Avaliação

A avaliação experimental é dividida em duas partes. Na primeira, o MSExtractor é aplicado a quatro sistemas legados *open source*: JPStore, SpringBlog, JForum e Roller. Os sistemas variam de 24 a 534 classes e permitem avaliar o comportamento da abordagem em aplicações de diferentes tamanhos. Os resultados são comparados com três outras abordagens de extração de microserviços: FoME, MEM e LIMBO [1].

Na segunda parte, os autores utilizam três projetos originalmente desenvolvidos segundo uma arquitetura de microserviços. A decomposição original desses sistemas funciona como *ground truth*. Os componentes são considerados conjuntamente como uma aplicação monolítica e o MSExtractor é executado para verificar em que medida consegue reconstruir a separação original dos serviços. A comparação utiliza principalmente as medidas de precisão e revocação. Não há

participantes humanos nessa etapa experimental; a comparação das decomposições é realizada de forma automática [1].

C. Resultados

Na comparação com FoME, MEM e LIMBO, o MSExtractor apresenta resultados favoráveis na maioria das métricas analisadas pelos autores, especialmente nos sistemas maiores. No Roller, que possui 534 classes, por exemplo, o número de interações registrado pela métrica IRN é 946 para o MSExtractor, 1.441 para o FoME e 2.786 para o MEM, sendo menores valores preferíveis para essa métrica [1].

A comparação entre algoritmos também fornece evidências em favor do IBEA no contexto avaliado. O algoritmo apresenta valores de hipervolume superiores aos obtidos por NSGA-II e SPEA2 nos quatro sistemas utilizados. As diferenças são estatisticamente significativas segundo o teste de Wilcoxon ($p < 0,05$), com tamanhos de efeito predominantemente grandes. Os autores concluem, portanto, que o IBEA se mostrou uma escolha mais adequada para o problema e para os sistemas avaliados [1].

Na comparação com o *ground truth*, a precisão obtida pelo MSExtractor varia entre 0,73 e 0,91, enquanto a revocação varia entre 0,61 e 0,74. Os autores investigam a revocação inferior e identificam dificuldades relacionadas a classes quase idênticas que não deveriam ser agrupadas. Em arquiteturas de microserviços podem existir classes redundantes que representam o mesmo tipo de dado, mas são adaptadas às necessidades de cada serviço e, inclusive, ao tipo de banco de dados utilizado pelo respectivo microserviço [1].

Esse resultado é particularmente relevante para o foco deste trabalho. O banco de dados aparece na análise dos autores como parte da explicação de determinados erros de recuperação, mas **não participa dos objetivos que orientam a busca**. Portanto, o MSExtractor evidencia uma limitação importante: uma decomposição orientada exclusivamente por dependências do código pode não capturar diferenças arquiteturais relacionadas à persistência dos dados. Os próprios autores destacam, ainda, a importância da validação humana das decomposições produzidas automaticamente [1].

VII. SAMIM – KHOSHNEVIS (2023)

Khoshnevis [2] apresenta o SAMIM, uma abordagem baseada em busca para identificação de microserviços variáveis em aplicações SaaS. O problema tratado difere daquele abordado pelo MSExtractor: o objetivo é identificar microserviços considerando diferentes necessidades dos clientes de uma aplicação SaaS, incorporando explicitamente a variabilidade existente entre suas possíveis configurações.

A entrada do SAMIM não é diretamente o código-fonte de uma aplicação monolítica. O método recebe um *Business Process Family Model* (BPFM), ou Modelo de Família de Processos de Negócio, que representa de forma estruturada diferentes configurações possíveis de um processo de negócio [2].

A. Processo do SAMIM

O funcionamento do SAMIM é organizado em três fases principais: **pré-identificação**, **identificação** e **pós-identificação**. Na primeira fase, o BPFM é transformado em representações intermediárias utilizadas pelas etapas posteriores. Na segunda, um algoritmo de otimização multiobjetivo procura agrupamentos das atividades em microserviços. Finalmente, na pós-identificação, a solução otimizada é combinada às informações de variabilidade para produzir uma Arquitetura de Microserviços com Variabilidade (*Variable Microservice Architecture* – VMA) [2].

A Figura 3 sintetiza esse fluxo. O modelo configurável representa as diferentes necessidades dos clientes SaaS; a etapa central aplica otimização multiobjetivo; e o resultado é uma arquitetura capaz de representar microserviços comuns e variáveis.

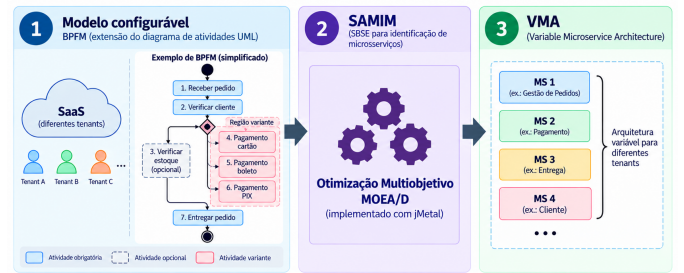


Figura 3. Processo geral do SAMIM. Fonte: adaptado de Khoshnevis [2].

A Figura 3 evidencia uma diferença fundamental em relação ao MSExtractor: a busca não parte diretamente das classes de uma aplicação. O espaço de soluções é construído a partir das atividades de um modelo de processo configurável e de sua variabilidade. Assim, a identificação dos microserviços procura preservar características compartilhadas e variáveis entre diferentes configurações SaaS [2].

B. Entrada: BPFM e predefinição

O BPFM é uma representação configurável de processos de negócio baseada em elementos de Diagramas de Atividades UML. Ele pode representar atividades obrigatórias e opcionais, pontos de variação, atividades variantes e regiões variantes. Uma única família de processos pode, portanto, representar diversas configurações válidas que correspondem a diferentes necessidades dos clientes SaaS [2].

Antes da otimização, o SAMIM executa uma etapa de predefinição que produz duas representações intermediárias: a *Single Weighted Configuration* (SWC) e o *Configuration Set* (CS).

A SWC resume a variabilidade do BPFM por meio de pesos associados às atividades. Para cada atividade a , o valor $p(a)$ representa sua ocorrência nas configurações válidas. Assim, uma atividade presente em todas as configurações recebe maior peso que uma atividade presente somente em parte delas. Essa representação permite que a otimização considere

a variabilidade sem precisar avaliar separadamente todas as configurações do modelo [2].

O CS, por sua vez, preserva o conjunto das configurações válidas derivadas do BPFM. Enquanto a SWC sintetiza a frequência das atividades, o CS mantém quais combinações efetivamente podem ocorrer. Essa informação volta a ser utilizada na etapa posterior à otimização para construir a VMA [2].

C. Otimização multiobjetivo com MOEA/D

Na fase de identificação, o SAMIM utiliza o MOEA/D (*Multi-Objective Evolutionary Algorithm based on Decomposition*). O algoritmo trabalha com uma população de soluções e decompõe o problema multiobjetivo em subproblemas associados a diferentes combinações dos objetivos [2].

Cada solução candidata é representada por um vetor de inteiros. A posição corresponde a uma atividade do BPFM e o valor identifica o microserviço ao qual a atividade foi atribuída. Por exemplo,

$$[1, 1, 2, 2]$$

para as atividades A , B , C e D representa a decomposição

$$MS_1 = \{A, B\}, \quad MS_2 = \{C, D\}.$$

Portanto, assim como no exemplo geral de espaço de busca apresentado anteriormente, cada cromossomo representa uma **decomposição completa**, e não um único microserviço.

A população inicial é gerada aleatoriamente e contém 500 soluções. O processo evolutivo é executado durante 10.000 gerações. Novas soluções são produzidas por seleção por torneio binário, cruzamento por evolução diferencial, com taxa de 0,95, e mutação polinomial, aplicada com probabilidade de 0,01 [2].

Durante esse processo, cada cromossomo é avaliado simultaneamente segundo três objetivos: **convergência de entidades de negócio**, **granularidade** e **comunalidade**.

D. Objetivos de otimização

A **convergência de entidades de negócio** procura agrupar atividades relacionadas às mesmas entidades de negócio. O cálculo utiliza uma matriz Entidade–Atividade, na qual os acessos são diferenciados entre leitura (r), escrita (w) e leitura/escrita (r/w), associados respectivamente aos valores 0,5, 1 e 1,5. O peso $p(a)$ proveniente da SWC também participa do cálculo, fazendo com que a frequência da atividade nas configurações seja considerada [2].

Esse objetivo introduz uma dimensão de dados diferente daquela observada no MSExtractor. Embora o SAMIM não opere diretamente sobre tabelas e chaves estrangeiras de um banco físico, ele considera explicitamente a relação entre atividades e **entidades de negócio** acessadas por elas.

O segundo objetivo é a **granularidade**. O SAMIM procura uma granularidade intermediária, evitando tanto decomposições excessivamente grossas quanto excessivamente fragmentadas. Para isso, o método estabelece um valor-alvo de

granularidade, denominado GG , calculado como a média de todas as razões de atividades por microserviço possíveis:

$$GG = \frac{1}{n} \sum_{k=1}^n \frac{n}{k} = \sum_{k=1}^n \frac{1}{k}$$

em que n é o número de atividades. Para doze atividades, por exemplo, $GG \approx 3,10$. O método procura minimizar a distância da solução em relação a esse alvo [2].

O terceiro objetivo é a **comunalidade**. Um microserviço é considerado comum quando contém pelo menos uma atividade obrigatória. O grau de comunalidade é definido por

$$VCOMM(x) = \frac{cms}{ms},$$

em que cms representa a quantidade de microserviços comuns e ms a quantidade total de microserviços. O trabalho estabelece 0,5 como valor-alvo, buscando um equilíbrio entre serviços comuns e variáveis [2].

E. Seleção da solução e construção da VMA

Após as 10.000 gerações, a otimização produz um conjunto de soluções não dominadas. Para selecionar uma solução desse conjunto, o SAMIM emprega a função de Tchebycheff com pesos iguais para os três objetivos e ponto ideal $[0, 0, 0]$. A função escalariza os objetivos procurando minimizar o maior desvio em relação ao ponto ideal [2]. Por exemplo, entre duas soluções candidatas $A = [0,2; 0,1; 0,3]$ e $B = [0,7; 0,4; 0,1]$, o maior desvio de A em relação ao ponto ideal é 0,3 e o de B é 0,7; portanto, A seria selecionada.

A solução selecionada e o conjunto de configurações CS são então fornecidos ao *VMA Builder*. Essa etapa classifica os microserviços como obrigatórios ou opcionais e identifica relações de inclusão (*includes*) e exclusão (*excludes*) entre microserviços opcionais. O resultado é a VMA, que representa tanto os serviços identificados quanto sua variabilidade [2].

F. Avaliação experimental

A avaliação utiliza seis casos experimentais provenientes de três empresas relacionadas aos domínios de manufatura e vendas. Os casos correspondem a processamento de pedido de compra, processamento de pedidos, processamento de ordem de reparo, compras online, submissão de cartas e emissão de notas contábeis [2].

Foram inicialmente contatadas 556 pessoas. Após a aplicação dos critérios estabelecidos pelos autores, 27 participantes qualificados participaram da avaliação. O grupo incluía profissionais da indústria, estudantes de graduação e pós-graduação e professores com conhecimentos relacionados a arquitetura e projeto de software, SaaS, microserviços, SOA e variabilidade [2].

Os participantes realizaram manualmente a identificação dos microserviços para os casos fornecidos. As soluções humanas foram então comparadas à solução produzida pelo SAMIM. A avaliação considera duas perspectivas: uma comparação **escalar**, baseada nos valores das métricas, e uma comparação **estrutural**, relacionada aos agrupamentos efetivamente produzidos.

G. Resultados

A comparação apresenta similaridade escalar média de 71,57% e similaridade estrutural média de 42,95%. Esses valores representam perspectivas diferentes: a similaridade escalar indica proximidade entre os valores das métricas, enquanto a estrutural compara a organização efetiva dos elementos em microsserviços. Portanto, 42,95% não deve ser interpretado como percentual de “acerto” da arquitetura [2].

Em determinados casos e métricas, a solução do SAMIM também apresenta distância até 57,85% menor em relação ao valor-alvo que a média das soluções humanas. Esse resultado se refere à proximidade de uma métrica específica de seu objetivo, e não significa que a arquitetura produzida pelo algoritmo seja, de forma geral, 57,85% superior às arquiteturas humanas [2].

A Figura 4 apresenta a comparação escalar da convergência entre entidades e atividades obtida pelo SAMIM e pela média dos especialistas.

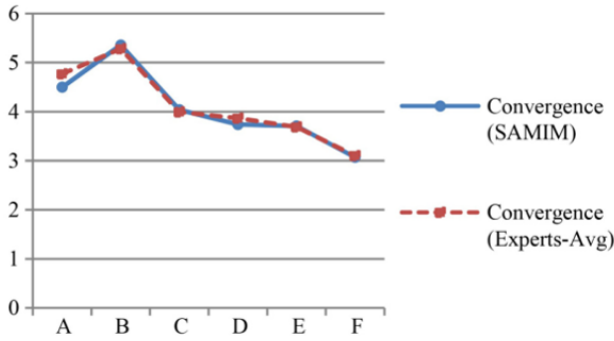


Figura 4. Comparação da convergência entre entidades de negócio e atividades para os seis casos experimentais. Fonte: Khoshnevis [2].

A convergência mede a relação entre atividades agrupadas e as entidades de negócio que elas acessam. A proximidade entre os valores obtidos pelo SAMIM e pelos especialistas varia entre os casos, o que reforça que a similaridade escalar média não implica igualdade estrutural entre as decomposições [2].

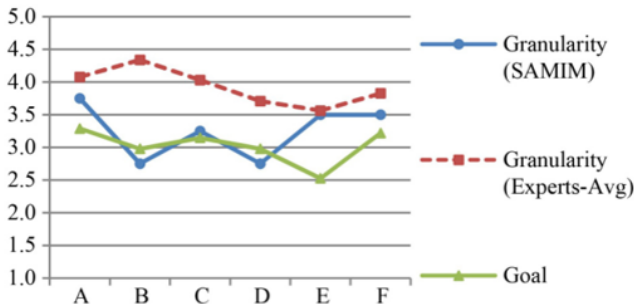


Figura 5. Comparação dos valores de granularidade obtidos pelo SAMIM e pelos especialistas nos seis casos experimentais. Fonte: Khoshnevis [2].

Como mostra a Figura 5, a interpretação da granularidade é realizada em relação à distância para o valor-alvo. Portanto,

o objetivo não é simplesmente maximizar ou minimizar a granularidade, mas produzir uma decomposição cuja granularidade se aproxime do valor considerado adequado para cada caso. A comparação permite observar separadamente o comportamento do SAMIM, da média dos especialistas e da referência utilizada pela função objetivo [2].

A comunalidade avalia uma dimensão diferente. No SAMIM, ela corresponde à proporção de microsserviços comuns em relação ao total de microsserviços da solução. O método estabelece 0,5 como valor-alvo, buscando um equilíbrio entre microsserviços comuns e variáveis [2]. A Figura 6 apresenta a comparação entre os resultados do SAMIM, a média dos especialistas e esse valor de referência.

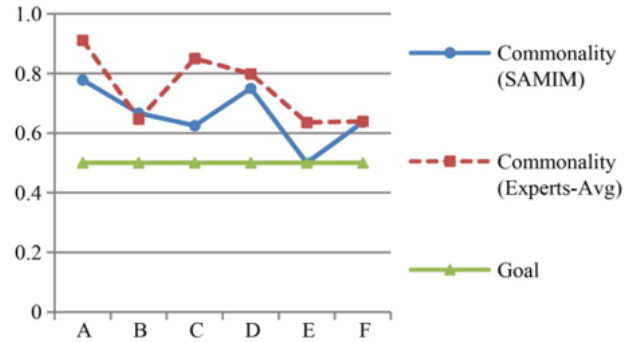


Figura 6. Comparação dos valores de comunalidade obtidos pelo SAMIM e pelos especialistas nos seis casos experimentais. Fonte: Khoshnevis [2].

Como mostra a Figura 6, o valor-alvo da comunalidade permanece constante em 0,5 nos seis casos experimentais. Os resultados apresentados por Khoshnevis [2] mostram que as soluções produzidas pelo SAMIM permanecem mais próximas desse objetivo do que a média das soluções dos especialistas nos casos avaliados. Assim como na granularidade, o resultado deve ser interpretado pela proximidade ao valor-alvo estabelecido pela função objetivo, e não pela simples maximização da métrica.

Em síntese, o SAMIM amplia a perspectiva da identificação de fronteiras ao considerar não apenas agrupamentos, mas também variabilidade e relações entre atividades e entidades de negócio. Contudo, sua dimensão de dados permanece em um nível conceitual: são consideradas **entidades de negócio e seus acessos**, e não diretamente tabelas, colunas ou operações sobre o esquema físico do banco.

VIII. ABORDAGEM COMPLEMENTAR: MSDC – KINOSHITA E KANUKA (2024)

Kinoshita e Kanuka [3] propõem uma abordagem de decomposição de microsserviços baseada em otimização multiobjetivo. O trabalho é apresentado neste relatório como análise complementar aos dois estudos de SBSE. Embora o artigo discuta abordagens baseadas em busca em sua fundamentação, os autores denominam a proposta como método de otimização multiobjetivo e não afirmam diretamente que ela seja SBSE.

Diferentemente das abordagens anteriores, o método não procura produzir uma decomposição a partir do zero: ele parte de uma **decomposição inicial**, denominada MSDC, criada pelo arquiteto de TI, e gera alternativas que reduzem violações de políticas arquiteturais.

Nesse contexto, um MSDC corresponde a uma **solução candidata completa**: uma determinada decomposição do sistema monolítico em microserviços. Durante a otimização, diferentes MSDCs são gerados, avaliados e comparados [3].

A. Entrada e representação do MSDC

O método recebe como entrada um MSDC inicial e uma lista de exceções de violações. O MSDC inicial é definido pelo arquiteto de TI a partir das funções de negócio da aplicação. A lista de exceções também é especificada pelo arquiteto e permite registrar chamadas entre microserviços consideradas aceitáveis, evitando que o algoritmo tente corrigir relações intencionais [3].

Para representar a decomposição, os autores definem um metamodelo composto por doze elementos fundamentais: *API Group*, *API Gateway*, *API*, *microserviço*, *classe*, *método*, *banco de dados*, *tabela*, *coluna*, *chamada de API*, *chamada de método* e *acesso a dados por operações CRUD (Create, Read, Update e Delete)* [3].

As relações entre esses elementos permitem representar tanto a estrutura do código quanto a estrutura e os acessos aos dados. Microserviços contêm classes, classes contêm métodos, APIs chamam métodos, métodos podem chamar outros métodos e os acessos CRUD relacionam métodos às colunas do banco de dados. O método também representa transações a partir das sequências de métodos, colunas e acessos a dados executados durante uma chamada de API [3].

Essa representação torna o trabalho particularmente relevante para a discussão desenvolvida neste relatório, pois o banco de dados deixa de aparecer apenas indiretamente e passa a constituir parte explícita da solução candidata.

B. Políticas de decomposição

A avaliação de cada MSDC é orientada por quatro políticas de decomposição definidas pelos autores [3].

A primeira política, **P1**, estabelece que cada microserviço deve corresponder a uma única capacidade de negócio. As funções de negócio são previamente definidas pelo arquiteto e utilizadas como referência para verificar se um serviço reúne elementos pertencentes a funções diferentes.

A política **P2** trata das chamadas entre microserviços. Apenas chamadas previamente especificadas pelo arquiteto são consideradas aceitáveis. Chamadas adicionais entre serviços caracterizam violações porque introduzem dependências não previstas na decomposição.

A política **P3** atua diretamente sobre o banco de dados: cada tabela deve receber acesso de escrita de apenas um microserviço. O objetivo é evitar que diferentes serviços compartilhem responsabilidade de escrita sobre a mesma estrutura de persistência [3].

Finalmente, a política **P4** procura evitar transações distribuídas entre microserviços. Uma transação que atravessa fronteiras de serviços representa uma dependência entre eles e aumenta a complexidade necessária para manter a consistência dos dados [3].

P3 e P4 tornam explícita a consideração da dimensão de dados no processo de otimização, diferentemente de abordagens baseadas exclusivamente nas dependências do código.

C. Violações e funções de avaliação

Para cada política é definida uma função de avaliação: P1 corresponde a EF1, P2 a EF2, P3 a EF3 e P4 a EF4. De forma geral, essas funções expressam a proporção dos elementos envolvidos em determinada violação em relação ao total de elementos daquele tipo considerados na solução [3].

Cada MSDC pode, portanto, ser representado pelo vetor

$$[EF1, EF2, EF3, EF4].$$

O problema é formulado para minimizar simultaneamente as quatro funções. Quanto menores seus valores, menor é a ocorrência das violações correspondentes. Esses quatro valores também permitem comparar as soluções segundo dominância e identificar os MSDCs não dominados.

D. Geração de novos MSDCs

Após identificar as violações, o método aplica operações de correção específicas. Cada correção produz uma nova decomposição candidata e uma mesma violação pode admitir diferentes formas de correção, originando vários novos MSDCs [3].

As operações são predefinidas pelos autores e vinculadas ao tipo de violação [3]:

- para V1, o microserviço é dividido segundo as funções de negócio;
- para V2, podem-se unir os microserviços envolvidos, criar um novo serviço com os métodos dependentes do chamador e do chamado, ou criar um serviço contendo o método chamado;
- para V3, podem-se unir todos os serviços que escrevem na tabela, dividir a tabela entre serviços, ou criar um novo serviço com a tabela e todos os métodos que nela escrevem;
- para V4, podem-se unir todos os serviços envolvidos na transação, ou criar um novo serviço com os métodos e tabelas que recebem escrita nessa transação.

Essas transformações funcionam como operadores de vizinhança específicos do domínio: em vez de alterar arbitrariamente uma solução, geram decomposições que procuram remover uma violação arquitetural. O próprio artigo contrasta essa estratégia com a movimentação aleatória de uma função para outro microserviço, utilizada em trabalhos anteriores e capaz de produzir muitas soluções com pouco significado arquitetural [3].

Depois da geração, o vetor de objetivos é calculado para cada novo MSDC. Uma ordenação não dominada identifica os candidatos Pareto-ótimos, que são mantidos para a próxima

iteração. O ciclo volta então à detecção de violações e à aplicação de novas correções.

Segundo a descrição apresentada no artigo, os novos candidatos são produzidos pelas operações de correção arquitetural e a seleção automática é realizada por ordenação não dominada [3]. Os autores não informam, nessa descrição, o emprego ou a ausência de um algoritmo genético.

As Figuras 7 e 8, utilizadas na apresentação, ilustram esse processo iterativo.

Antes da primeira iteração existe um MSDC de entrada. A identificação das violações permite aplicar diferentes correções, produzindo múltiplos candidatos. Depois da avaliação multiobjetivo, apenas as soluções não dominadas permanecem.

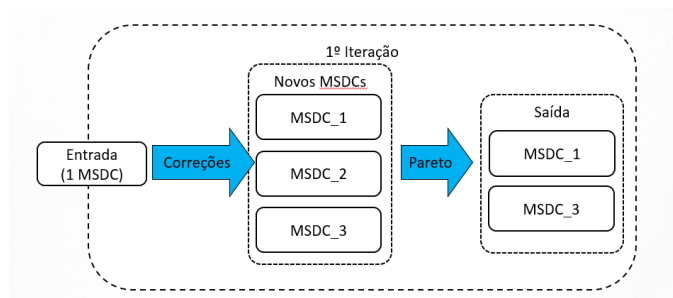


Figura 7. Exemplo da primeira iteração do processo de geração e seleção de MSDCs. Fonte: elaboração própria com base em Kinoshita e Kanuka [3].

A Figura 7 evidencia que a aplicação de uma correção não produz necessariamente uma única solução. Diferentes alternativas podem ser geradas a partir da mesma entrada e são posteriormente comparadas segundo as quatro funções de avaliação.

Na iteração seguinte, cada solução Pareto-ótima preservada pode originar novos candidatos, como ilustrado na Figura 8.

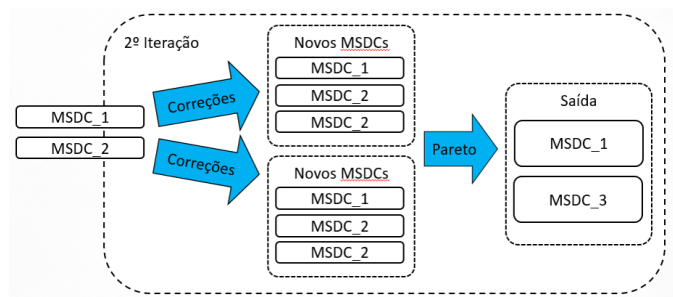


Figura 8. Exemplo da segunda iteração do processo de geração e seleção de MSDCs. Fonte: elaboração própria com base em Kinoshita e Kanuka [3].

A segunda iteração deixa claro o caráter progressivo do procedimento: as soluções não dominadas de uma iteração tornam-se pontos de partida para novas operações de correção. Dessa maneira, novas decomposições são produzidas por transformações arquiteturais sucessivas.

E. Natureza da busca e limites da otimização

Elementos explicitados no artigo. Kinoshita e Kanuka [3] distinguem abordagens automáticas de decomposição baseadas

em agrupamento e em busca e associam estas últimas à obtenção de soluções próximas do ótimo para um problema de particionamento considerado NP-difícil. Na apresentação do método proposto, os autores afirmam que ele amplia uma abordagem de otimização multiobjetivo. O problema é formulado matematicamente como a minimização simultânea de $EF1-EF4$, e cada MSDC é mapeado para um ponto no espaço definido por essas quatro funções.

O procedimento parte de um MSDC inicial e de uma lista de exceções, ambos fornecidos pelo arquiteto. Em cada iteração, o artigo descreve três etapas: identificar todas as violações de P1–P4 presentes nos MSDCs armazenados; aplicar as operações de correção correspondentes para gerar automaticamente novos MSDCs; e utilizar ordenação não dominada para preservar os candidatos classificados como Pareto-ótimos. Os candidatos preservados voltam a alimentar a iteração seguinte. O número de iterações é predeterminado, e os autores recomendam que seja superior à quantidade de violações do MSDC inicial [3].

As operações de correção são definidas previamente e vinculadas aos tipos de violação. Quando uma violação admite mais de uma operação, cada alternativa produz um novo candidato. O próprio artigo contrasta essa geração orientada por políticas com uma estratégia empregada em trabalho anterior, na qual uma função é escolhida aleatoriamente e movida para outro microserviço. Segundo os autores, restringir as transformações às correções procura reduzir a geração de soluções com pouco significado semântico [3].

Depois das iterações, linhas de referência são distribuídas no espaço dos quatro objetivos e utilizadas para escolher uma quantidade de MSDCs previamente determinada pelo arquiteto. O conceito de linhas de referência é relacionado pelos autores ao NSGA-III, mas o procedimento publicado descreve algoritmos próprios para ordenação não dominada, geração das linhas e seleção por distância. O artigo não afirma que o NSGA-III seja executado como algoritmo de busca no método proposto [3].

Dedução a partir desses elementos. As operações de correção podem ser interpretadas como operadores de vizinhança: elas transformam uma solução corrente em um conjunto de soluções alcançáveis. As funções $EF1-EF4$ fornecem o vetor de objetivos, e a ordenação não dominada decide quais candidatos continuam no processo. A repetição desse ciclo caracteriza uma exploração orientada de alternativas e oferece fundamento para compreender o método como baseado em busca e otimização, ainda que sua mecânica seja diferente dos algoritmos evolutivos empregados no MSExtractor e no SAMIM.

Essa interpretação não autoriza concluir que o método seja genético. O artigo não declara se a implementação utiliza ou não algoritmo genético e, na descrição do método proposto, não apresenta população evolutiva, cruzamento, mutação ou seleção genética. Também não descreve sorteio na detecção das violações, na aplicação das correções ou na ordenação dos MSDCs. Assim, pode-se afirmar que **esses mecanismos não constam na descrição publicada**; não se pode afirmar

categoricamente que estejam ausentes de uma implementação não detalhada.

A aplicação automática de regras predefinidas, o número fixado de iterações e a construção matemática das linhas de referência sugerem ainda um procedimento determinístico para uma mesma entrada. Novamente, trata-se de dedução. O artigo não especifica a ordem de aplicação quando há várias violações, a eliminação de candidatos duplicados, a resolução de empates ou todos os detalhes da implementação. Portanto, não demonstra que execuções ou implementações diferentes necessariamente produzam o mesmo conjunto de MSDCs.

Limites da otimização. O método não percorre todas as partições possíveis: ele explora as decomposições alcançáveis a partir do MSDC inicial pelas correções associadas a P1–P4. O MSDC inicial funciona, inclusive, como um parâmetro que condiciona a geração das soluções. Além disso, candidatos não dominados são preservados e os demais são descartados a cada iteração. Os próprios autores reconhecem que essa eliminação antecipada pode conduzir a soluções localmente ótimas [3]. Portanto, os MSDCs chamados de Pareto-ótimos são não dominados no conjunto efetivamente gerado pelo procedimento; o estudo não oferece garantia de que representem a fronteira de Pareto global de todas as decomposições possíveis.

Essa limitação é coerente com o objetivo declarado do trabalho. O método procura evitar candidatos semanticamente pouco úteis, reduzir o conjunto apresentado e apoiar o arquiteto na comparação de alternativas. A restrição da busca por políticas e correções sacrifica cobertura do espaço em favor de significado arquitetural e tratabilidade. A solução final ainda pode ser refinada manualmente, inclusive porque somente quatro políticas são modeladas e outros critérios relevantes podem depender do julgamento do arquiteto [3].

F. Seleção por linhas de referência

O processo iterativo pode produzir um conjunto Pareto-ótimo com muitas soluções, criando um novo problema para o arquiteto: escolher entre diversas alternativas não dominadas. Para reduzir esse conjunto, Kinoshita e Kanuka [3] utilizam **linhas de referência**.

As linhas de referência representam diferentes direções no espaço formado pelas quatro funções de avaliação. Elas permitem selecionar soluções representativas de diferentes regiões da fronteira de Pareto e, consequentemente, preservar alternativas com características distintas.

O arquiteto determina previamente quantas soluções deseja receber. O método utiliza então as linhas de referência para selecionar essa quantidade de MSDCs representativos. Ao final, as soluções selecionadas são apresentadas juntamente com informações explicativas, permitindo ao arquiteto escolher uma alternativa e, se necessário, refiná-la [3].

Portanto, assim como nos demais trabalhos, a otimização atua como apoio à decisão. A escolha arquitetural final permanece sob responsabilidade do arquiteto.

G. Avaliação experimental

A avaliação utiliza três aplicações com características e tamanhos distintos. A primeira é o *Pet Clinic*, aplicação *open*

source baseada em Spring Boot, com aproximadamente mil linhas de código. A segunda é uma aplicação de *Internet Banking*, com aproximadamente 10 mil linhas de código e tecnologias como Struts, Spring, MySQL e iBatis. A terceira é uma aplicação de gerenciamento de contratos de seguros, com aproximadamente 20 mil linhas de código, baseada em Spring MVC, PostgreSQL e MyBatis [3].

Na configuração experimental, o processo foi executado por até 100 iterações. Ao final, foram obtidos 4 MSDCs para o *Pet Clinic*, 220 para o *Internet Banking* e 243 para a aplicação de seguros. Esses números mostram que, especialmente nas aplicações maiores, a otimização pode produzir um conjunto expressivo de alternativas, reforçando a necessidade da etapa posterior de seleção [3].

A avaliação é estruturada em cinco itens, EI1–EI5, que analisam diferentes aspectos do método: geração e diversidade das decomposições, tempo de geração, comportamento da seleção das soluções, relação entre os vetores de referência e as características dos MSDCs selecionados e, finalmente, o uso das informações produzidas pelo método por arquitetos de TI experientes [3].

H. Principais resultados

Os experimentos mostram que as linhas de referência permitem selecionar soluções com características relacionadas às direções especificadas no espaço dos objetivos. Na análise correspondente ao EI4, os autores observam correlação entre componentes dos vetores de referência e características das soluções selecionadas, embora os resultados variem entre as aplicações [3].

Na etapa envolvendo arquitetos experientes, correspondente ao EI5, as informações fornecidas pelo método foram utilizadas para priorizar políticas, selecionar uma solução e realizar seu refinamento. Esse refinamento ocorre **depois** da otimização e permite incorporar julgamento arquitetural sobre aspectos que não são completamente representados pelas quatro funções de avaliação [3].

O trabalho de Kinoshita e Kanuka é especialmente relevante para a dimensão de banco de dados discutida neste relatório. O metamodelo representa explicitamente bancos, tabelas, colunas e acessos CRUD; P3 avalia compartilhamento de escrita sobre tabelas; e P4 considera transações que atravessam fronteiras de microsserviços. Dessa maneira, informações relacionadas à persistência participam diretamente da avaliação e da geração das decomposições, e não apenas da análise posterior dos resultados [3].

IX. ANÁLISE COMPARATIVA

Os dois trabalhos centrais aplicam explicitamente SBSE multiobjetivo à identificação de fronteiras de microsserviços, enquanto o terceiro apresenta uma técnica complementar de otimização multiobjetivo. A comparação não pretende colocar as três propostas sob o mesmo rótulo, mas examinar suas diferenças quanto ao problema de origem, à representação das soluções, aos objetivos, à geração de alternativas e ao papel dos dados.

O MSExtractor apresenta a formulação mais diretamente orientada ao código: classes são agrupadas segundo dependências estruturais, e os objetivos procuram equilibrar coesão, acoplamento e granularidade. Essa formulação permite analisar sistemas legados sem exigir previamente um modelo de negócio, mas não representa a persistência. A análise de seus resultados mostra que decisões relacionadas aos dados podem explicar divergências em relação à arquitetura de referência [1].

O SAMIM desloca a análise para o nível dos processos de negócio. Sua principal contribuição para o conjunto estudado é considerar a variabilidade entre clientes SaaS e relacionar atividades a entidades de negócio. Os dados, portanto, influenciam a otimização, mas ainda como entidades conceituais e tipos de acesso, sem representar diretamente o esquema físico de persistência [2].

O método de Kinoshita e Kanuka parte de uma decisão arquitetural inicial e procura melhorá-la. Seu metamodelo integra funções de negócio, chamadas, elementos de código e estruturas de banco de dados. Como consequência, problemas como escrita compartilhada sobre tabelas e transações distribuídas tornam-se funções explícitas de avaliação. Em contrapartida, o método depende de um MSDC inicial detalhado, de informações fornecidas pelo arquiteto e de um repertório predefinido de correções [3].

Não se pode concluir, apenas a partir dos experimentos publicados, que uma abordagem seja globalmente superior às demais. Elas tratam entradas, objetivos e contextos diferentes e foram avaliadas com desenhos experimentais distintos. A comparação evidencia tanto a diversidade interna de SBSE — código no MSExtractor e processos de negócio no SAMIM — quanto a possibilidade de contrastá-la com uma otimização arquitetural orientada por políticas, correções e elementos físicos de persistência no MSDC.

X. CONCLUSÃO

A elaboração do seminário permitiu retomar e consolidar conceitos de SBSE abordados em aula, aplicando-os à análise de um problema concreto de Engenharia de Software: a identificação de fronteiras de microsserviços. A partir do estudo e da comparação dos três trabalhos, foi possível observar diferentes formas de modelar o problema, representar soluções, definir objetivos e conduzir o processo de otimização.

As discussões realizadas após a apresentação também tiveram papel importante nesse processo. O debate possibilitou compartilhar diferentes interpretações, revisitar conceitos e terminologias que ainda não estavam plenamente consolidados e aprofundar pontos que, embora parecessem claros durante a preparação, demandaram explicações adicionais durante a discussão. Esse exercício contribuiu não apenas para esclarecer aspectos dos trabalhos apresentados, mas também para identificar onde a própria compreensão e a forma de comunicar determinados conceitos poderiam ser aprimoradas. Além disso, as contribuições dos professores, ao relacionarem os temas discutidos a outros conceitos, fatos e experiências, ampliaram

a análise e proporcionaram um aprofundamento que foi além do conteúdo inicialmente previsto para o seminário.

A apresentação de artigos sobre temas correlatos exigiu o estudo de conceitos, técnicas e terminologias em seus contextos de aplicação, permitindo compreender não apenas seus significados, mas também como são empregados na solução de problemas concretos de Engenharia de Software. Essa perspectiva tornou mais evidentes as diferenças e aproximações entre os trabalhos analisados.

Mesmo entre os dois estudos explicitamente baseados em busca, os universos são muito diferentes. O MSExtractor recebe classes e dependências de código de aplicações legadas, representa decomposições por vetores e utiliza IBEA para otimizar coesão, acoplamento e granularidade. O SAMIM recebe processos de negócio configuráveis, representa agrupamentos de atividades e emprega MOEA/D para otimizar convergência, granularidade e comunalidade em aplicações SaaS. Eles diferem nas entradas, na modelagem, nos objetivos, nos operadores, na execução e na forma de selecionar a solução final. Isso evidencia que SBSE não corresponde a um algoritmo específico, mas a uma maneira de formular e explorar problemas de Engenharia de Software.

O MSDC acrescenta um contraste importante. Ele recebe uma decomposição inicial e uma lista de exceções, representa código, APIs e persistência e gera candidatos por correções predefinidas para violações de quatro políticas. Os candidatos são comparados por um vetor de funções de avaliação, ordenação não dominada e linhas de referência. Esses elementos permitem relacionar a proposta a busca e otimização; contudo, essa relação é uma interpretação fundamentada, não uma classificação explícita dos autores. O artigo não descreve os mecanismos evolutivos presentes nos dois estudos de SBSE e não fornece detalhes suficientes para afirmar se a implementação é genética, aleatória ou completamente determinística.

Entretanto, nota-se que toda essa diversidade está sustentada por uma mesma espinha dorsal: espaço de busca, representação das soluções, funções objetivo, geração de candidatos, avaliação, seleção e exploração de *trade-offs*. Um problema de Engenharia de Software é inicialmente formulado como um problema de otimização. As possíveis decisões passam a constituir um **espaço de busca**, no qual cada solução candidata precisa ser representada de uma forma que possa ser manipulada pelo método. Sobre essa representação são definidas **funções objetivo**, responsáveis por traduzir propriedades desejáveis da solução em valores que permitam sua avaliação e comparação. A partir daí, algum mecanismo de busca ou transformação explora esse espaço, produzindo novos candidatos e direcionando o processo segundo os objetivos estabelecidos.

Nos problemas multiobjetivo analisados, essa estrutura torna especialmente visível outro fundamento importante: raramente existe uma solução que seja simultaneamente a melhor em todos os critérios. Melhorar coesão, acoplamento, granularidade, convergência, comunalidade ou outros atributos pode produzir objetivos conflitantes. A otimização passa, portanto, pela exploração desses *trade-offs*, pela comparação entre soluções

e pela identificação de alternativas **não dominadas**, formando ou aproximando uma **fronteira de Pareto**. O resultado deixa de ser necessariamente uma única resposta “ótima” e passa a ser um conjunto de alternativas que expressa diferentes compromissos entre os objetivos.

É justamente nesse nível de abstração que as diferenças entre os trabalhos ajudam a consolidar o fundamento comum. **Mudam a entrada, a representação da solução, as funções objetivo, os operadores, o algoritmo e os critérios utilizados para selecionar uma solução; permanece, contudo, a ideia de transformar uma decisão de Engenharia de Software em um problema que possa ser sistematicamente representado, avaliado e explorado.** No MSExtractor e no SAMIM, essa estrutura aparece explicitamente como SBSE e é operacionalizada por algoritmos evolutivos. No MSDC, embora o artigo não classifique a proposta como SBSE, vários desses elementos reaparecem em uma otimização guiada por políticas, correções arquiteturais e avaliação multiobjetivo.

Assim, mais do que identificar qual algoritmo cada trabalho utiliza, **o estudo permitiu compreender o que permanece por trás dessas diferentes implementações:** a modelagem do problema determina o que constitui uma solução, a função de avaliação determina o que significa uma solução melhor e o mecanismo de busca determina como o espaço de alternativas será explorado. A otimização organiza e torna explícita essa exploração e seus *trade-offs*, mas não elimina a decisão de Engenharia de Software. A escolha e a validação finais continuam dependentes do contexto do sistema, das prioridades envolvidas e do julgamento do arquiteto.

REFERÊNCIAS

- [1] K. Sellami, A. Ouni, M. A. Saied, S. Bouktif, and M. W. Mkaouer, “Improving microservices extraction using evolutionary search,” *Information and Software Technology*, vol. 151, p. 106996, 2022. doi: 10.1016/j.infsof.2022.106996.
- [2] S. Khoshnevis, “A search-based identification of variable microservices for enterprise SaaS,” *Frontiers of Computer Science*, vol. 17, no. 3, p. 173208, 2023. doi: 10.1007/s11704-022-1390-4.
- [3] T. Kinoshita and H. Kanuka, “Enhancing automated microservice decomposition via multi-objective optimization,” *IEEE Access*, vol. 12, pp. 55697–55710, 2024. doi: 10.1109/ACCESS.2024.3389700.